

Содержание

Глава 1. Теоретико-методологические основы исследования	3
Глава 2. Механизм построения решения	5
2.1. Концепция механизма	5
2.1.1 Формализация решения	5
2.1.2 Предрешение игры	5
2.1.3 Распределение дополнительных выплат	7
2.2. Исследование механизма	8
2.2.1 Решение системы	8
2.2.2 Предрешение по эксцессам	10
2.2.3 Сложность полного прохода по коалициям	12
2.2.4 Свойства предрешений	15
Глава 3. Алгоритмы построения предрешения игры	16
3.1. Описание алгоритмов	16
3.1.1 Базовый алгоритм	16
3.1.2 Оптимизированный алгоритм	16
3.1.3 Альтернативный алгоритм	18
3.2. Формализация алгоритмов	19
3.2.1 Базовый алгоритм	19
3.2.2 Оптимизированный алгоритм	19
3.2.3 Альтернативный алгоритм	20
3.3. Программная реализация	22
Глава 4. Тестирование алгоритмов	25
4.1. Подготовка к тестам	25
4.1.1 MSCP	25
4.1.2 Генерация игр	25
4.1.3 Поиск N-ядра	26
4.1.4 Метрики	27
4.2. Тесты на сгенерированных играх	28
4.2.1 Время работы	28
4.2.2 Превышение минимальной суммы	28

4.2.3	Процент попаданий в С-ядро	29
4.2.4	Гарантия попадания в С-ядро	30
4.2.5	Отклонение от N-ядра	32
4.2.6	Отклонение от вектора Шепли	32
Выводы		33
Заключение		33
Список литературы		34

Глава 1. Теоретико-методологические основы исследования

Определение. Кооперативной игрой [3] называется (N, v) , где $N = \{1, 2, \dots, n\}$ – множество игроков;
 $v : 2^N \rightarrow \mathbb{R}$ – характеристическая функция игры, задающая выигрыш каждой коалиции $S \subseteq N$, при этом $v(\emptyset) = 0$.

Определение. Коалицией S называется множество игроков из 2^N : $S \in 2^N$.

Определение. Максимальная коалиция – это коалиция, включающая всех игроков: $S = N$.

Обозначение. Пусть α – некоторый вектор распределения выигрыша, тогда

$$\alpha(S) = \sum_{i \in S} \alpha_i, \quad \forall S \subseteq N.$$

Определение. Дележом[3] называется вектор распределения выигрыша $\alpha = (\alpha_1, \dots, \alpha_n) \in \mathbb{R}^n$, удовлетворяющий следующим условиям:

$\alpha(N) = v(N)$ – эффективность;

$\alpha_i \geq v(\{i\})$, $i \in N$ – индивидуальная рациональность.

Определение. Решением будем называть правило, которое каждой игре сопоставляет единственный вектор из множества дележей.

Определение. Будем говорить, что распределение $x \in \mathbb{R}^n$ является устойчивым относительно коалиции S , если выполняется

$$x(S) \geq v(S).$$

Замечание. Если распределение x^T для коалиции T является устойчивым относительно всех коалиций $S \subset T$, то ни одна коалиция S не может выйти из T и перераспределить свой выигрыш $v(S)$ таким образом, чтобы все её участники получили больше.

Определение. С-ядро [3] – множество дележей, устойчивых относительно всех коалиций игроков

$$x \in C(N, v) \neq \emptyset \Leftrightarrow x(S) \geq v(S), \quad \forall S \subseteq N.$$

Определение. Сбалансированной игрой будем называть такую игру, в которой существует непустое С-ядро.

Определение. Набор коалиций $S = \{S_1, S_2, \dots, S_m\}$ называется сбалансированным, если

$$\exists \lambda_{S_i} > 0 : \sum_{S_i \ni j} \lambda_{S_i} = 1, \quad \forall j \in N.$$

Определение. Эксцессом [7] будем называть разницу между выигрышем коалиции и сумой соответствующих компонент из заданного распределения

$$e(S, x) = v(S) - x(S).$$

Глава 2. Механизм построения решения

2.1 Концепция механизма

2.1.1 Формализация решения

Представим искомое решение в виде вектора

$$x = \tilde{x} + \Delta v,$$

где

- $\tilde{x} = (\tilde{x}_1, \dots, \tilde{x}_n)^\top$ – вектор неполного распределения выигрыша, который является устойчивым относительно всех коалиций, кроме максимальной, назовём его *предрешением игры*;
- $\Delta v = (\Delta v_1, \dots, \Delta v_n)^\top$ – вектор дополнительных выплат, сумма его компонент равна разнице между выигрышем максимальной коалиции и *предрешением игры*, т. е.

$$\Delta v(N) = v(N) - \tilde{x}(N).$$

Искомое решение при этом будет удовлетворять условию эффективности, т. к.

$$\sum_{i \in N} x_i = \sum_{i \in N} \tilde{x}_i + \sum_{i \in N} \Delta v_i = \tilde{x}(N) + \Delta v(N) = \tilde{x}(N) + [v(N) - \tilde{x}(N)] = v(N).$$

Таким образом, выплата каждого участника представляется в виде суммы некоторой минимальной выплаты и разницы, распределённой по выбранному правилу

$$x_i = \tilde{x}_i + \Delta v_i.$$

2.1.2 Предрешение игры

Для каждой коалиции, кроме размера 2, будем решать систему

$$Ax^S = v^S,$$

где $S = \{s_1, \dots, s_k\}$ – выбранная коалиция, s_i – игрок, находящийся на позиции i внутри коалиции S , k – размер коалиции S , т.е. $k = |S|$, $k \neq 2$ и

- $(k \times k)$ -матрица A состоит из вложенных коалиций размера $k - 1$, где каждая строка – это бинарный вектор участия игроков соответствующей коалиции,

$$A = \begin{pmatrix} a_{11} & \cdots & a_{1k} \\ \vdots & \ddots & \vdots \\ a_{k1} & \cdots & a_{kk} \end{pmatrix}, \quad a_{ij} = \begin{cases} 1, & i \neq j \\ 0, & i = j \end{cases};$$

- v^S – вектор выигрышей коалиций, где индекс компонента равен номеру строки, соответствующей коалиции в матрице A ,

$$v^S = (v(S \setminus \{s_1\}), \dots, v(S \setminus \{s_k\}))^\top;$$

- x^S – распределение, которое получается в результате решения системы, где индекс компоненты соответствует отсутствующему игроку в сопоставленной коалиции, будем называть его *предрешением по коалиции S* .

Предрешение x^S обеспечивает равенство

$$x^S(S \setminus \{i\}) = v(S \setminus \{i\}), \quad \forall i \in S.$$

Тогда, если построить распределение $\tilde{x}$, где каждая компонента будет равна наибольшему значению выплаты соответствующего игрока среди всех полученных предрешений x^S , а также собственного выигрыша, то будет выполнено

$$\tilde{x}(S) \geq v(S), \quad \forall S \subset N,$$

т. е, $\tilde{x}$ гарантирует устойчивость относительно всех коалиций, кроме максимальной, что и требуется от *предрешения игры*.

Замечание 1. Все наборы коалиций, используемые для построения предрешения по коалиции, являются сбалансированными. Для набора

$$S = \{S_1, \dots, S_k\}$$

коалиций размера $k - 1$ каждой коалиции соответствует

$$\lambda_{S_i} = \frac{1}{k-1}.$$

Тогда для любого игрока j выполняется $\sum_{S_i \ni j} \lambda_{S_i} = 1$.

На этапе фиксации размера коалиции k наборы также являются сбалансированными, а каждой коалиции соответствует

$$\lambda_{S_i} = \frac{1}{k-1}.$$

Замечание 2. Поскольку матрица A невырожденная при $k \geq 2$, система

$$Ax^S = v^S,$$

имеет единственное решение вида $x^S = A^{-1}v^S$.

2.1.3 Распределение дополнительных выплат

Заметим, что предрешение никак не зависит от $v(N)$, то есть изменение выигрыша максимальной коалиции никак не влияет на компоненты предрешения. Рассмотрим, как изменение $v(N)$ повлияет на вектор Шепли.

Для этого в формуле компоненты вектора Шепли

$$\varphi_i(v) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(n - |S| - 1)!}{n!} (v(S \cup \{i\}) - v(S))$$

выделим из общей суммы, только часть, зависящую от $v(N)$

$$\frac{|N \setminus \{i\}|!(n - |N \setminus \{i\}| - 1)!}{n!}(v(N) - v(N \setminus \{i\})) \sim \frac{v(N)}{n} = \frac{v(N)}{|N|}$$

и подставим в разность компонент

$$\varphi_i(\bar{v}) - \varphi_i(v) = \frac{(\bar{v}(N) - v(N))}{|N|}.$$

Таким образом, при изменении выигрыша максимальной коалиции, вектор Шепли распределяется разницу равномерно между игроками.

Следуя его примеру, можно распределять вектор дополнительных выплат равномерно между игроками, то есть по правилу

$$\Delta v(N) = \frac{v(N) - \tilde{x}(N)}{|N|}.$$

Тогда, при изменении выигрыша максимальной коалиции, разность также будет распределяться равномерно

$$x_i(\bar{v}) - x_i(v) = \left(\tilde{x}_i + \frac{\bar{v}(N) - \tilde{x}(N)}{|N|} \right) - \left(\tilde{x}_i + \frac{v(N) - \tilde{x}(N)}{|N|} \right) = \frac{\bar{v}(N) - v(N)}{|N|}.$$

2.2 Исследование механизма

2.2.1 Решение системы

Обратим матрицу A методом Гаусса

$$A|E = \left(\begin{array}{cccc|cccc} 0 & 1 & \cdots & 1 & 1 & 0 & \cdots & 0 \\ 1 & 0 & \cdots & 1 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & \cdots & 0 & 0 & 0 & \cdots & 1 \end{array} \right) \sim$$

$$\begin{aligned}
& \sim \left(\begin{array}{cccccc|cccccc} -1 & 1 & 0 & \cdots & 0 & 0 & 1 & -1 & 0 & \cdots & 0 & 0 \\ 0 & -1 & 1 & \cdots & 0 & 0 & 0 & 1 & -1 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & -1 & 1 & 0 & 0 & 0 & \cdots & 1 & -1 \\ 1 & 1 & 1 & \cdots & 1 & 0 & 0 & 0 & 0 & \cdots & 0 & 1 \end{array} \right) \sim \\
& \sim \left(\begin{array}{cccccc|cccccc} -1 & 1 & 0 & \cdots & 0 & 0 & 1 & -1 & 0 & \cdots & 0 & 0 \\ 0 & -1 & 1 & \cdots & 0 & 0 & 0 & 1 & -1 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & -1 & 1 & 0 & 0 & 0 & \cdots & 1 & -1 \\ 0 & 0 & 0 & \cdots & 0 & k-1 & 1 & 1 & 1 & \cdots & 1 & -(k-2) \end{array} \right) \sim \\
& \sim \left(\begin{array}{cccccc|ccccc} -1 & 0 & \cdots & 0 & 0 & \frac{k-2}{k-1} & -\frac{1}{k-1} & \cdots & -\frac{1}{k-1} & -\frac{1}{k-1} \\ 0 & -1 & \cdots & 0 & 0 & -\frac{1}{k-1} & \frac{k-2}{k-1} & \cdots & -\frac{1}{k-1} & -\frac{1}{k-1} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & -1 & 0 & -\frac{1}{k-1} & -\frac{1}{k-1} & \cdots & \frac{k-2}{k-1} & -\frac{1}{k-1} \\ 0 & 0 & \cdots & 0 & 1 & \frac{1}{k-1} & \frac{1}{k-1} & \cdots & \frac{1}{k-1} & -\frac{k-2}{k-1} \end{array} \right) \sim \\
& \sim \left(\begin{array}{cccc|cccc} 1 & 0 & \cdots & 0 & -\frac{k-2}{k-1} & \frac{1}{k-1} & \cdots & \frac{1}{k-1} \\ 0 & 1 & \cdots & 0 & \frac{1}{k-1} & -\frac{k-2}{k-1} & \cdots & \frac{1}{k-1} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 & \frac{1}{k-1} & \frac{1}{k-1} & \cdots & -\frac{k-2}{k-1} \end{array} \right) = E|A^{-1}.
\end{aligned}$$

Теперь посчитаем предрешение

$$\begin{aligned}
x^S &= A^{-1}v^S = \\
&= \begin{pmatrix} -\frac{k-2}{k-1} & \frac{1}{k-1} & \cdots & \frac{1}{k-1} \\ \frac{1}{k-1} & -\frac{k-2}{k-1} & \cdots & \frac{1}{k-1} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{1}{k-1} & \frac{1}{k-1} & \cdots & -\frac{k-2}{k-1} \end{pmatrix} \begin{pmatrix} v(S \setminus \{s_1\}) \\ v(S \setminus \{s_2\}) \\ \vdots \\ v(S \setminus \{s_k\}) \end{pmatrix} = \\
&= \begin{pmatrix} \frac{-(k-2)v(S \setminus \{s_1\}) + v(S \setminus \{s_2\}) + \cdots + v(S \setminus \{s_k\})}{k-1} \\ \frac{v(S \setminus \{s_1\}) - (k-2)v(S \setminus \{s_2\}) + \cdots + v(S \setminus \{s_k\})}{k-1} \\ \vdots \\ \frac{v(S \setminus \{s_1\}) + v(S \setminus \{s_2\}) + \cdots - (k-2)v(S \setminus \{s_k\})}{k-1} \end{pmatrix} =
\end{aligned}$$

$$= \begin{pmatrix} \frac{-(k-1)v(S \setminus \{s_1\}) + \sum_{i \in S} v(S \setminus \{s_i\})}{k-1} \\ \frac{-(k-1)v(S \setminus \{s_2\}) + \sum_{i \in S} v(S \setminus \{s_i\})}{k-1} \\ \vdots \\ \frac{-(k-1)v(S \setminus \{s_k\}) + \sum_{i \in S} v(S \setminus \{s_i\})}{k-1} \end{pmatrix} = \begin{pmatrix} \frac{\sum_{i \in S} v(S \setminus \{s_i\})}{k-1} - v(S \setminus \{s_1\}) \\ \frac{\sum_{i \in S} v(S \setminus \{s_i\})}{k-1} - v(S \setminus \{s_2\}) \\ \vdots \\ \frac{\sum_{i \in S} v(S \setminus \{s_i\})}{k-1} - v(S \setminus \{s_k\}) \end{pmatrix}$$

или в упрощённом виде

$$x_j^S = \frac{\sum_{i \in S} v(S \setminus \{s_i\})}{|S| - 1} - v(S \setminus \{s_j\}), \quad \forall S, \forall j \in S.$$

2.2.2 Предрешение по эксцессам

Пусть будет известно некоторое распределение x^0 , ограничивающее выигрыш игроков снизу. Вычислим эксцессы коалиций

$$e(S \setminus \{i\}, x^0) = v(S \setminus \{i\}) - \sum_{k \in S \setminus \{i\}} x_k^0, \quad \forall i \in S.$$

Их сумма будет равна

$$\sum_{i \in S} e(S \setminus \{i\}, x^0) = \sum_{i \in S} [v(S \setminus \{i\}) - (\sum_{k \in S} x_k^0 - x_i^0)] = \sum_{i \in S} v(S \setminus \{i\}) - (|S| - 1) \sum_{i \in S} x_i^0.$$

Теперь подставим эксцессы в формулу предрешения по коалиции

$$\begin{aligned} x_i^{e(S, x^0)} &= \frac{\sum_{k \in S} e(S \setminus \{k\}, x^0)}{|S| - 1} - e(S \setminus \{i\}, x^0) = \\ &= \frac{\sum_{k \in S} v(S \setminus \{k\}) - (|S| - 1) \sum_{k \in S} x_k^0}{|S| - 1} - v(S \setminus \{i\}) - (\sum_{k \in S} x_k^0 - x_i^0) = \\ &= \frac{\sum_{k \in S} v(S \setminus \{k\})}{|S| - 1} - v(S \setminus \{i\}) - x_i^0 = x_i^S - x_i^0. \end{aligned}$$

Отсюда видно, что предрешение можно считать и по эксцессам, если известно ограничивающее снизу распределение.

Рассмотрим случай, в котором некоторые собственные выигрыши коа-

лиций могут принимать нулевые значения.

$$\text{Пусть } T \subset S \text{ и } \begin{cases} v(S \setminus \{i\}) = 0 & \forall i \in T; \\ v(S \setminus \{j\}) \neq 0 & \forall j \in S \setminus T, \end{cases}$$

тогда

$$\begin{aligned} x_i^S &= \frac{\sum_{k \in S} v(S \setminus \{k\})}{|S| - 1} - v(S \setminus \{i\}) = \frac{\sum_{k \in S \setminus T} v(S \setminus \{k\})}{|S| - 1}, \\ x_j^S &= \frac{\sum_{k \in S \setminus T} v(S \setminus \{k\})}{|S| - 1} - v(S \setminus \{j\}). \end{aligned}$$

Помимо того

$$\begin{aligned} \sum_{k \in S} x_k^S &= |S| \frac{\sum_{k \in S} v(S \setminus \{k\})}{|S| - 1} - \sum_{k \in S} v(S \setminus \{k\}) = \frac{\sum_{k \in S} v(S \setminus \{k\})}{|S| - 1} = \\ &= \frac{\sum_{k \in S \setminus T} v(S \setminus \{k\})}{|S| - 1}. \end{aligned}$$

Тогда

$$\sum_{k \in T} x_k^S = \frac{|T| \sum_{k \in S \setminus T} v(S \setminus \{k\})}{|S| - 1} = |T| \sum_{k \in S} x_k^S,$$

следовательно

$$\sum_{k \in S \setminus T} x_k^S = -\frac{(|T| - 1) \sum_{k \in S \setminus T} v(S \setminus \{k\})}{|S| - 1}.$$

Вывод: если уже известно некоторое ограничивающее распределение, можно считать предрешение по эксцессам коалиций. Более того, можно использовать только положительные эксцессы, а остальные замещать нулём, так как относительно них ограничивающее распределение уже устойчиво.

Сформулируем на основе изученных свойств подход и назовём его *предрешение по эксцессам коалиции S*.

Пусть задано ограничивающее распределение x^0 . Вычислим эксцессы

$$ev(S \setminus \{i\}) = v(S \setminus \{i\}) - \sum_{k \in S} x_k^0, \quad \forall i \in S.$$

Сохраняем игроков, без которых эксцессы коалиций становятся нулевыми или отрицательными.

$$T = \{i \mid e(S \setminus \{i\}) \leq 0, i \in S\}.$$

Посчитаем общий член в формуле предрешения

$$term = \frac{\sum_{k \in S \setminus T} e(S \setminus \{k\})}{|S| - 1}.$$

Вычисляем предрешение по эксцессам коалиции

$$x_i^S = x_i^0 + \begin{cases} term, & \forall i \in T \\ \max\{0, term - e(S \setminus \{i\})\}, & \forall i \in S \setminus T. \end{cases}$$

2.2.3 Сложность полного прохода по коалициям

С учётом упрощённой формулы для вычисления предрешений

$$x_j^S = \frac{\sum_{i \in S} v(S \setminus \{s_i\})}{|S| - 1} - v(S \setminus \{s_j\})$$

можно оценить асимптотическую сложность вычисления предрешений всех коалиций.

Для построения предрешения по фиксированной коалиции S нужно сложить выигрыши всех подкоалиций на единицу меньшей размерности ($|S| - 1$ операций), разделить полученную сумму на $|S| - 1$ (1 операция) и из результата отдельно для каждой компоненты предрешения вычесть выигрыш коалиции без соответствующего выбранной компоненте игрока ($|S|$ операций). Итого, $2|S|$ операций для каждой коалиции, размера больше 2, при

полном переборе. То есть,

$$\sum_{k=3}^{|N|} 2kC_{|N|}^k$$

операций.

Упростим формулу, используя бином Ньютона

$$(a+b)^n = \sum_{k=0}^n a^k b^{n-k} C_n^k.$$

Подставим $a = x, b = 1$, тогда

$$(x+1)^n = \sum_{k=0}^n x^k C_n^k$$

и продифференцируем по x

$$n(x+1)^{n-1} = \sum_{k=0}^n kx^{k-1} C_n^k.$$

Умножим на x

$$xn(x+1)^{n-1} = \sum_{k=0}^n kx^k C_n^k$$

и, подставляя $x = 1$, получим

$$n2^{n-1} = \sum_{k=0}^n kC_n^k.$$

Пусть $|N| = n$, тогда, используя полученную формулу, имеем

$$\sum_{k=3}^n 2kC_n^k = 2(n2^{n-1} - C_n^1 - 2C_n^2),$$

учитывая $C_n^1 = n, 2C_n^2 = n(n-1)$, получаем

$$2(n2^{n-1} - n - n(n-1)) = n2^n - 2n^2$$

операций, откуда следует асимптотическая сложность

$$O(n2^n).$$

Сравним с вектором Шепли [6]

$$\varphi_i(v) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(n - |S| - 1)!}{n!} (v(S \cup \{i\}) - v(S)).$$

Часть

$$\frac{|S|!(n - |S| - 1)!}{n!}$$

является общей для всех коалиций одной размерности и может быть предварительно вычислена для каждой размерности коалиций, поэтому не влияет на асимптотическую сложность.

С учётом этого, остаётся 1 умножение, 1 вычитание и 1 сложение (кроме самого первого в сумме) для каждой коалиции, не включающей i игрока. Таких коалиций

$$2^{n-1},$$

то есть

$$3 \cdot 2^{n-1} - 1$$

операций для каждого игрока и

$$3n2^{n-1} - n,$$

для всех игроков, что немного больше предложенного метода, но различие лишь в константах, а не в порядке роста

$$O(n2^n).$$

Вычисление N-ядра [8] является значительно более трудной задачей: не известно алгоритма с гарантированной оценкой сложности, сопоставимой с $O(n2^n)$.

Вывод: Сложность полного прохода совпадает со сложностью вычис-

ления вектора Шепли — $O(n2^n)$ и значительно меньше, чем у вычисления N-ядра, для которого не известно алгоритма с сопоставимой гарантированной оценкой.

2.2.4 Свойства предрешений

1. $v(S \setminus \{j\}) = \sum_{k \in S} v(S \setminus \{k\}) - x_j^S = x^S(S \setminus \{j\});$
2. $v(S) = \sum_{k \in S} x_k^S + \Delta v^S = v(S \setminus \{j\}) + x_j^S + \Delta v^S;$
3. $v(S) - v(S \setminus \{j\}) = x_j^S + \Delta v^S;$
4. $x_j^S = \frac{\sum_{k \in S} v(S \setminus \{k\})}{|S| - 1} - v(S \setminus \{s_j\});$
5. $\sum_{k \in S} x_k^S = |S| \frac{\sum_{k \in S} v(S \setminus \{k\})}{|S| - 1} - \sum_{k \in S} v(S \setminus \{k\}) = \frac{\sum_{k \in S} v(S \setminus \{s_k\})}{|S| - 1};$
6. Если $\exists i \in S : v(S \setminus \{i\}) > 0$ и $\forall j \in S \setminus \{i\} : v(S \setminus \{j\}) = 0$, то

$$\sum_{k \in S} v(S \setminus \{k\}) = v(S \setminus \{i\}) \Rightarrow \begin{cases} x_i^S = -\frac{(|S| - 2)v(S \setminus \{i\})}{|S| - 1}; \\ x_j^S = \frac{v(S \setminus \{i\})}{|S| - 1}. \end{cases}$$

Глава 3. Алгоритмы построения предрешения игры

3.1 Описание алгоритмов

3.1.1 Базовый алгоритм

Алгоритм представляет собой примитивную реализацию — полный проход коалиций и построение *предрешения игры* по максимальным компонентам.

Плюсы:

- Стабильное время работы — количество операций зависит только от размера игры;
- Простота — отсутствие сложной логики делает алгоритм лёгким в реализации и предсказуемым в поведении и свойствах.

Минусы:

- Сравнительно низкая вероятность попадания в С-ядро — выбор максимальных компонент сильно завышает сумму компонент предрешения, из-за чего вероятность получить гарантию С-ядра низкая.
- Низкая скорость работы — вычисление предрешений по всем коалициям требует сравнительно большого числа операций.

3.1.2 Оптимизированный алгоритм

Одной из проблем базового алгоритма является необходимость пересчитывать предрешение по всем коалициям размера больше 2. Устойчивость относительно крупных коалиций влечёт устойчивость относительно более мелких, из-за чего необходимость их учитывать отпадает.

Таким образом, можно построить предрешение по максимальной коалиции, а затем с помощью динамического программирования рассчитывать эксцессы всех остальных, обновляя *предрешение игры* через *предрешение по*

эксцессам коалиции. Вычисление эксцессов через ДП (динамическое программирование) имеет асимптотическую сложность $O(2^n)$, и такой же будет обладать предложенный алгоритм в лучшем случае.

Но тогда возникает проблема с определением фиксируемой коалиции. После вычисления эксцессов образуется список неустойчивых коалиций, а для обновления *предрешения игры* нужно зафиксировать одну конкретную коалицию и из вложенных в неё неустойчивых коалиций вычислять *предрешение по эксцессам*.

Для ускорения процесса для каждого игрока можно посчитать сумму эксцессов неустойчивых коалиций, в которые он входит (в рамках коалиций одного размера), отсортировать полученный список в порядке невозрастания и из первых в списке игроков построить коалицию, по которой затем вычислять *предрешение по эксцессам*.

Иногда может возникнуть ситуация, что среди вложенных коалиций в коалицию, построенную таким образом, нет ни одной с положительным эксцессом. В таком случае её следует заменить на коалицию, состоящую из самой неустойчивой коалиции и самого неустойчивого игрока, среди тех, что не состоят в самой неустойчивой коалиции.

Плюсы:

- Сравнительно высокая скорость работы — асимптотическая сложность в лучшем случае может достигать $O(2^n)$ благодаря ДП и игнорированию устойчивых коалиций;
- Выше вероятность попадания в С-ядро — сумма компонент *предрешения* зачастую меньше, чем в базовом алгоритме, за счёт игнорирования устойчивых коалиций.

Минусы:

- Накладные вычисления — при большом количестве неустойчивых коалиций и их особой структуре может потребоваться большое количество дополнительных вычислений на пересчёт эксцессов и сумм для ДП;
- Приоритет на наиболее неустойчивых игроках — возможна некоторая нечестность в распределении, связанная с порядком прохода коалиций.

3.1.3 Альтернативный алгоритм

Попадание в С-ядро гарантируется при выполнении условия $\tilde{x}(N) \leq v(N)$. Тогда уменьшение $\tilde{x}(N)$ приведёт к увеличению вероятности попадания в С-ядро.

Если построить предрешение игры по минимальным значениям предрешений всех коалиций и заменить выигрыши коалиций на их эксцессы, то при повторном проходе по ним часть эксцессов, вероятно, станет отрицательными и их можно не учитывать при расчёте эксцессов по коалиции.

Проходя таким образом итеративно, на некотором шаге все коалиции станут устойчивыми. А собранное предрешение игры будет по сумме компонент меньше, чем предрешение игры базового алгоритма и, возможно, оптимизированного, поскольку эксцессы уменьшаются понемногу, а не за раз.

Поскольку по сложности алгоритм представляет собой множество проходов базового, даже с учётом того, что устойчивые коалиции исключаются из рассмотрения, разумно установить критерий остановки, а именно $\tilde{x}(N) \leq v(N)$, где вектор предрешения представляет собой сумму вектора максимального предрешения на текущей итерации и минимальных предрешений со всех предыдущих.

Плюсы:

- Выше вероятность попадания в С-ядро — сумма компонент предрешения зачастую меньше, чем в базовом алгоритме и, в некоторых случаях, чем в оптимизированном;
- Сравнительно более справедливое распределение выигрыша — из-за постепенного увеличения предрешения в первую очередь будет расти выигрыш тех игроков, кто внёс высокий вклад в наибольшее количество коалиций.

Минусы:

- Очень низкая скорость работы — зачастую требуется большое количество проходов по большому количеству коалиций.

3.2 Формализация алгоритмов

3.2.1 Базовый алгоритм

1. Вычислить *предрешения* для всех коалиций $|S| \geq 3$ по формуле

$$x_i^S = \frac{\sum_{j \in S} v(S \setminus \{j\})}{|S| - 1} - v(S \setminus \{i\}), \quad \forall i \in S;$$

2. Построить *предрешение игры* по правилу

$$\tilde{x}_i = \max \{v(\{i\}), x_i^S \mid S \subseteq N, i \in S, |S| \geq 3\}.$$

3.2.2 Оптимизированный алгоритм

1. Установить стартовое *предрешение* на основе индивидуальной рациональности

$$\tilde{x}_i = \max \{0, v(\{i\})\}, \quad \forall i \in N;$$

2. Вычислить *предрешение по эксцессам* максимальной коалиции и заменить им старое *предрешение*;
3. Вычислить *сумму* компонент *предрешения*;
4. Установить $k = |N| - 2$;
5. Для всех коалиций размера k вычислить *суммы* соответствующих компонент *предрешения*, посредством вычитания из известных ранее *сумм*;
6. При помощи полученных *сумм* составить список *эксцессов коалиций* $(e_S^+)_{|S|=k}$ и *эксцессов игроков* $(e_i^+)_{i \in N}$, но учитывать и сохранять лишь положительные эксцессы

$$e_i^+ = \sum_{\substack{S \ni i \\ |S|=k \\ v(S) > 0}} v(S), \quad i \in N;$$

7. Если в списке положительных *эксцессов коалиций* меньше двух элементов:
 - (а) Если в списке один элемент, эксцесс распределить равномерно между игроками, входящими в эту коалицию, и обновить *суммы*;
 - (b) Перейти к шагу 13.
8. Отсортировать список *эксцессов игроков* в порядке невозрастания и из первых $k + 1$ игроков сформировать коалицию.
9. Если в сформированной коалиции нет ни одной вложенной с положительным эксцессом, перестроить её — выбрать первую коалицию из списка *эксцессов коалиций* и добавить к ней первого игрока из списка *эксцессов игроков*, который не входит в выбранную коалицию.
10. По сформированной коалиции пересчитать *предрешение по эксцессам*, используя старое *предрешение* в качестве ограничивающего распределения;
11. Обновить *суммы*, список *эксцессов коалиций* и список *эксцессов игроков* для коалиций размера k ;
12. Перейти к шагу 7;
13. Уменьшить k на 1;
14. Если $k > 1$, перейти к шагу 5.

Результатом работы алгоритма является финальное *предрешение*, оно и будет *предрешением игры*.

3.2.3 Альтернативный алгоритм

1. Установить стартовое *предрешение* на основе индивидуальной рациональности

$$\tilde{x}_i = \max \{0, v(\{i\})\}, \quad \forall i \in N;$$

2. Приравнять *минимальное предрешение* стартовому предрешению и установить ∞ как *сумму лучшего предрешения* $\tilde{x}_{best}(N)$;
3. Создать список *положительных эксцессов* из всех коалиций размера $k = \overline{2, n-1}$;
4. Вычесть из списка *положительных эксцессов* компоненты *минимального предрешения* соответствующих игроков, удаляя из списка все неположительные эксцессы;
5. Если список *положительных эксцессов* оказался пустым, завершить работу алгоритма;
6. Вычислить *предрешения по эксцессам* всех коалиций $|S| \geq 3$, у которых есть хотя бы одна вложенная коалиция в списке *положительных эксцессов*, и построить *минимальное предрешение*

$$\tilde{x}_{mini} = \min \{ x_i^S \mid S \subseteq N, i \in S, |S| \geq 3 \}$$

и *максимальное предрешение*

$$\tilde{x}_{maxi} = \max \{ x_i^S \mid S \subseteq N, i \in S, |S| \geq 3 \};$$

7. Если

$$\tilde{x}(N) + \tilde{x}_{max}(N) \leq \tilde{x}_{best}(N),$$

переопределить *лучшее предрешение*

$$\tilde{x}_{besti} = \tilde{x}_i + \tilde{x}_{maxi};$$

8. Если $\tilde{x}_{best}(N) \leq v(N)$, завершить работу алгоритма;
9. Прибавить к вектору *предрешения* вектор *минимального предрешения*

$$\tilde{x} := \tilde{x} + \tilde{x}_{min};$$

10. Перейти к шагу 4.

Результатом работы алгоритма является *лучшее предрешение* $\tilde{x}_{best}$, оно же и будет *предрешением* игры.

3.3 Программная реализация

Весь код написан на языке Python в виде классов и прикреплён в приложении 1, 2 и 3 для базового, оптимизированного и альтернативного алгоритмов соответственно. Оптимизированный и альтернативный алгоритмы наследуют у базового инициализацию и все методы, но переопределяют метод `_run`. Метод `_run` отвечает за исполнение самого алгоритма и применяется во время инициализации.

При создании классы принимают параметры:

- `n` — множество игроков; задаётся последовательностью элементов, которые используются как идентификаторы игроков, либо целым числом `n` — тогда игроки идентифицируются от 1 до `n`;
- `v` — характеристическая функция; задаётся словарём, где ключи — последовательности из идентификаторов игроков в коалиции, а значения — выигрыши коалиций;
- `precision` — число знаков после запятой при выводе результата; задаётся целым числом;
- `eps` — погрешность при сравнении; задаётся целым числом, применяется как $10^{-\text{eps}}$;
- `early_stop` — останавливать ли вычисления при попадании в С-ядро (только у альтернативного алгоритма); задаётся булевым значением, по умолчанию `True`;

Взаимодействие с классом после создания:

- `x_N` — атрибут; хранит сумму предрешения игры;
- `delta_v` — атрибут; хранит сумму дополнительных выплат $\Delta v(N)$;

- `get_x` — метод, возвращающий вычисленное предрешение игры; возвращает словарь где ключами являются идентификаторы игроков;
- `get_imputation` — метод, возвращающий вычисленное решение; принимает последовательность `extra_imputation_form`, которая определяет пропорции распределения дополнительных выплат в порядке нумерации игроков, по умолчанию равномерное распределение; возвращает словарь, где ключами являются идентификаторы игроков;
- `check_excess` — метод, вычисляющий эксцессы всех коалиций, кроме грандкоалиции; параметр `check` задаёт распределение и принимает `"imputation"` – посчитанное решение, `"x"` – посчитанное предрешение или последовательность; параметр `"get_excess"` отвечает за то, возвращать ли ключи в виде идентификаторов, принимает булево значение; параметр `print_excesses` отвечает за то, выводить ли полный список эксцессов (в виде идентификаторов); возвращает словарь: `"in_core"` — булево значение — является ли распределение устойчивым относительно всех проверяемых коалиций, `"excess"` — словарь из эксцессов коалиций.

В качестве системы хранения собственных выигрышей коалиций используются битмаски. Каждый ключ словаря собственных выигрышей представляет собой целое число, то есть маску коалиции. Двоичное представление маски является бинарным вектором участия игроков в коалиции. Если игрок под номером i включён в коалицию, то в двоичном представлении маски на позиции i будет стоять единица, иначе 0. Нумерация игроков начинается с нуля. Тогда маска коалиции вычисляется по формуле

$$mask_S = \sum_{i \in S} 2^i, \quad \forall S \subseteq N.$$

Такое представление позволяет быстрее производить операции над множествами. Представление в виде встроенных в язык программирования множеств при любых изменениях требует пересобирать множество с нуля и пересчитывать хеш, а представление в виде битмасок сводит всё к дешёвым (по

количеству вычислений) побитовым операциям.

Примеры применения побитовых операций над битмасками:

- Объединение: $A \cup B \text{ — } A \mid B$;
- Пересечение: $A \cap B \text{ — } A \& B$;
- Исключение: $A \setminus B \text{ — } A \wedge B$.

Глава 4. Тестирование алгоритмов

4.1 Подготовка к тестам

4.1.1 MSCP

Была написана вспомогательная функция для поиска предрешения с минимально возможной суммой компонент — MSCP (сокращение от Min Sum Core Point). Она реализована как ЛП-минимизация с ограничениями в виде выигрышей всех коалиций, кроме максимальной. Для ЛП-минимизации использовался класс `method = "linprog"` из библиотеки «`scipy.optimize`» с параметром `method = "highs"`. `highs` автоматически выбирает метод между симплекс-методом и методом внутренней точки на основе размера задачи. Полученное решение зачастую является неединственным, однако сумма компонент полезна по нескольким причинам:

- Позволяет проверить существование С-ядра — игра сбалансирована только в случае, если выигрыш максимальной коалиции больше или равен этой сумме;
- Позволяет оценить насколько сумма компонент предрешения каждого предложенного алгоритма превысила возможный минимум.

4.1.2 Генерация игр

Написана функция для генерации случайных или супераддитивных игр. Функции передаётся `random.default_rng(seed)` из библиотеки `numpy`, где `seed` — целое число, обеспечивающее воспроизводимость: при одинаковом значении будут генерироваться одни и те же числа между сессиями. Затем генерируется массив из $v(S) \sim \text{Uniform}(0, |S|)$.

Если параметр `superadditive` принимает `True`, происходит коррекция супераддитивности снизу вверх по размеру коалиций. Для каждой коалиции S начиная с размера 2:

$$v(S) = \max \left(v(S), \max_{T_1 \sqcup T_2 = S} (v(T_1) + v(T_2)) \right),$$

где T_1 обходит все непустые собственные подмножества S , а T_2 достраивается как дополнение. Пары, где маски $T_1 \geq T_2$, пропускаются во избежание повторного счёта.

В конце к игре применяется MSCP и выигрышу максимальной коалиции присваивается сумма компонент MSCP, обеспечивая сбалансированность игры.

Перед тестами выигрыш максимальной коалиции обеих игр увеличивается на

$$\delta \sim \text{Uniform}(0, 2c \cdot v(N)),$$

где $c > 0$ — коэффициент, задающий математическое ожидание приращения $\mathbb{E}[\delta] = c \cdot v(N)$.

4.1.3 Поиск N-ядра

Известно, что поиск N-ядра сводится к последовательности ЗЛП:

Инициализация: активные коалиции $\mathcal{A} = \{S \subseteq N : \emptyset \subsetneq S \subsetneq N\}$.

На каждом шаге:

1. Решаем задачу линейного программирования с переменными $x_1, \dots, x_n \in \mathbb{R}$ и $\varepsilon \in \mathbb{R}$:

$$\min \varepsilon$$

при ограничениях

$$\begin{aligned} v(S) - \sum_{i \in S} x_i &\leq \varepsilon, \quad \forall S \in \mathcal{A}; \\ \sum_{i=1}^n x_i &= v(N); \end{aligned}$$

2. Получаем оптимальное ε^* и соответствующий вектор $x^* = (x_1^*, \dots, x_n^*)$;
3. Находим насыщенные коалиции:

$$\mathcal{F} = \left\{ S \in \mathcal{A} : v(S) - \sum_{i \in S} x_i^* = \varepsilon^* \right\};$$

4. Обновляем активные коалиции: $\mathcal{A} \leftarrow \mathcal{A} \setminus \mathcal{F}$;
5. Повторяем пока $\mathcal{A} \neq \emptyset$.

Результат: x^* из последней итерации — N-ядро.

4.1.4 Метрики

Для всех измерений вычисляется среднее выборочное по всем играм одного типа, размера и коэффициента c .

Список метрик:

- Время работы, в секундах;
- Превышение $\min x(N)$ — вычисляется по формуле

$$\frac{x(N) - x_{MSCP}(N)}{x_{MSCP}(N)} \cdot 100\%;$$

- Процент попаданий в С-ядро — процент случаев, когда все эксцессы оказывались неположительными;
- Процент гарантий попадания в С-ядро — процент случаев, когда выполнялось $\tilde{x}(N) \leq v(N)$;
- Процент отклонения решения от N-ядра — вычисляется по формуле

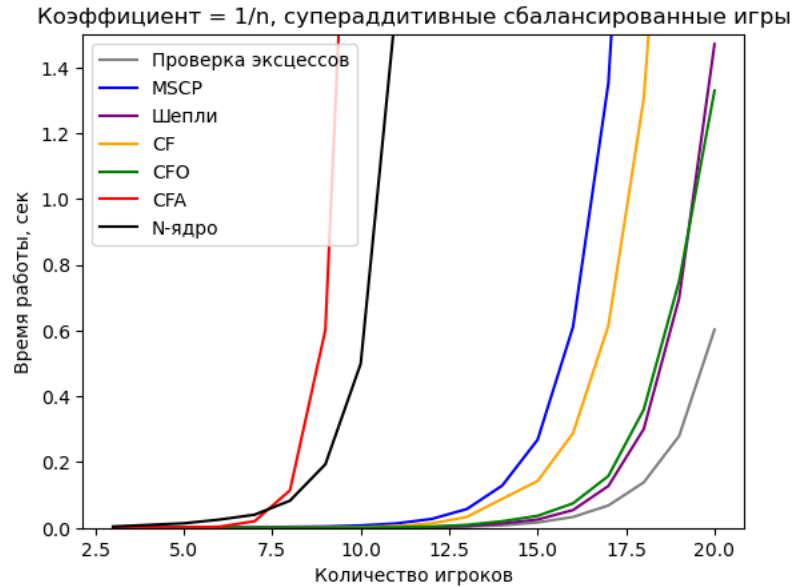
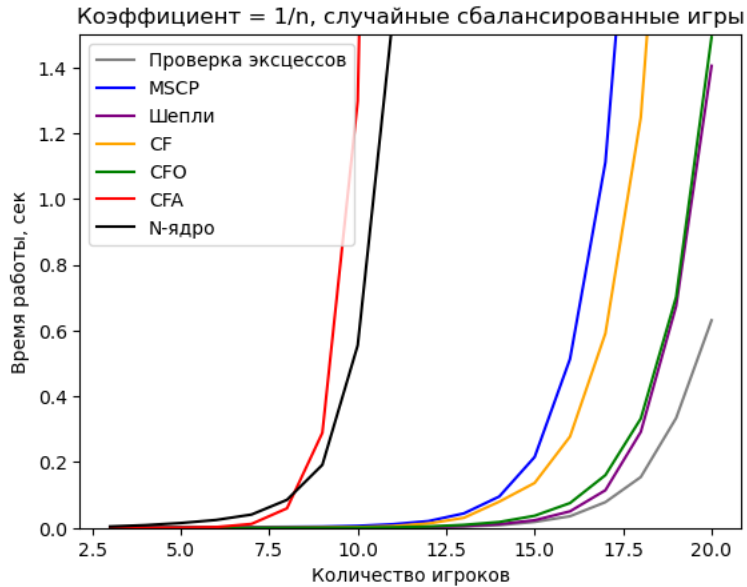
$$\frac{\sqrt{\sum_{i \in N} (x_i - x_{N\text{-ядро } i})^2}}{v(N)} \cdot 100\%;$$

- Процент отклонения решения от вектора Шепли — вычисляется по формуле

$$\frac{\sqrt{\sum_{i \in N} (x_i - \varphi_i)^2}}{v(N)} \cdot 100\%.$$

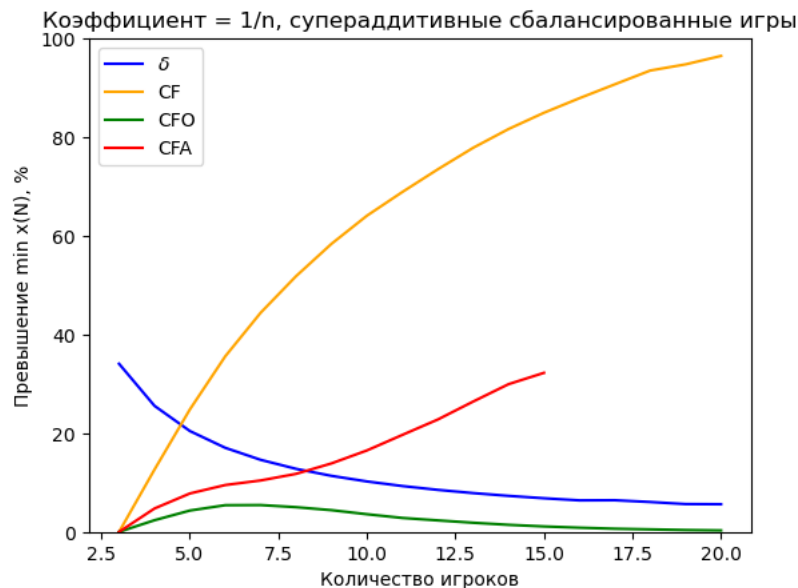
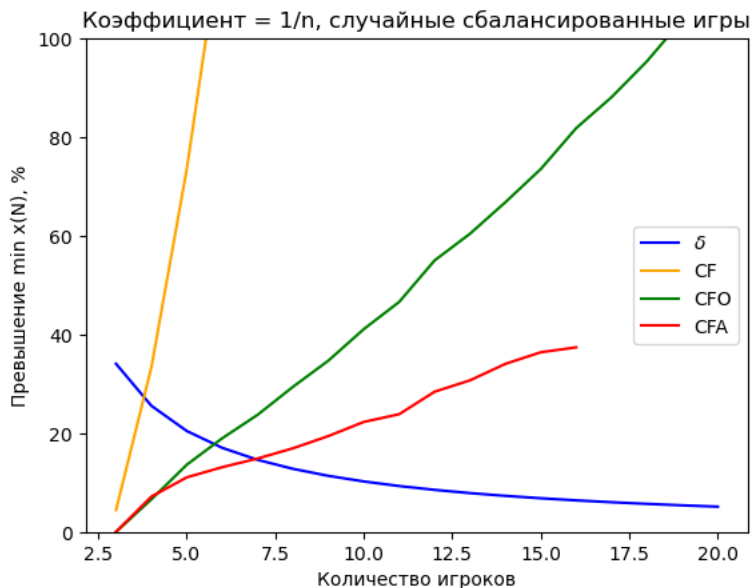
4.2 Тесты на сгенерированных играх

4.2.1 Время работы



- Оптимизированный алгоритм сопоставим по времени с вектором Шепли;
- Базовый алгоритм работает несколько быстрее ЛП-минимизации;
- Альтернативный алгоритм работает значительно медленнее остальных, даже немного медленнее, чем поиск N-ядра.

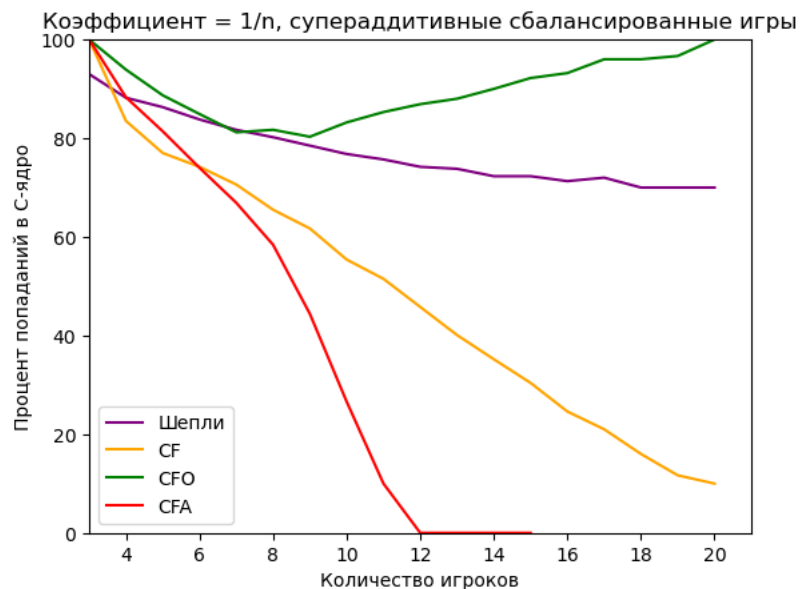
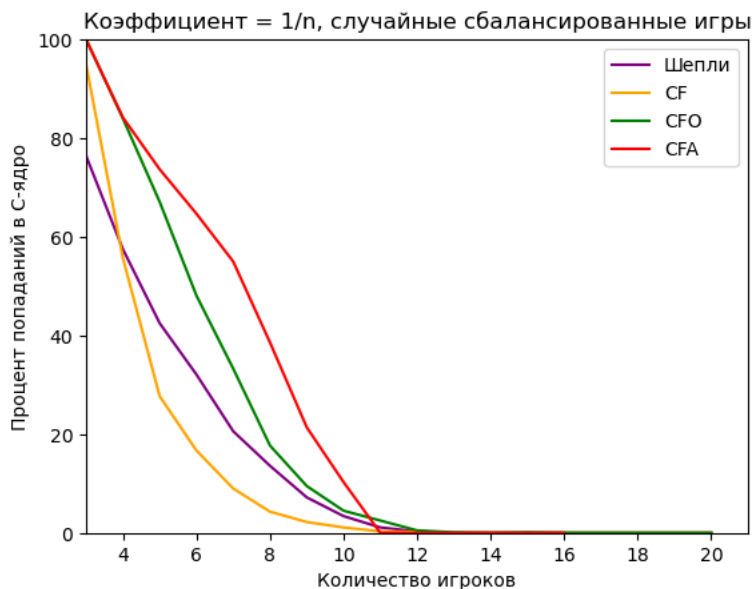
4.2.2 Превышение минимальной суммы



- Оптимизированный алгоритм показывает резкий линейный рост отклонения на случайных играх, однако стремится к минимуму на супераддитивных;
- Альтернативный алгоритм демонстрирует схожие результаты на обоих типах игр, с лучшим результатом на случайных;
- Базовый алгоритм обладает наибольшим ростом отклонения;
- На случайных играх демонстрируют значительное отклонение все три алгоритма, на супераддитивных не выходит за границу только оптимизированный алгоритм.

4.2.3 Процент попаданий в С-ядро

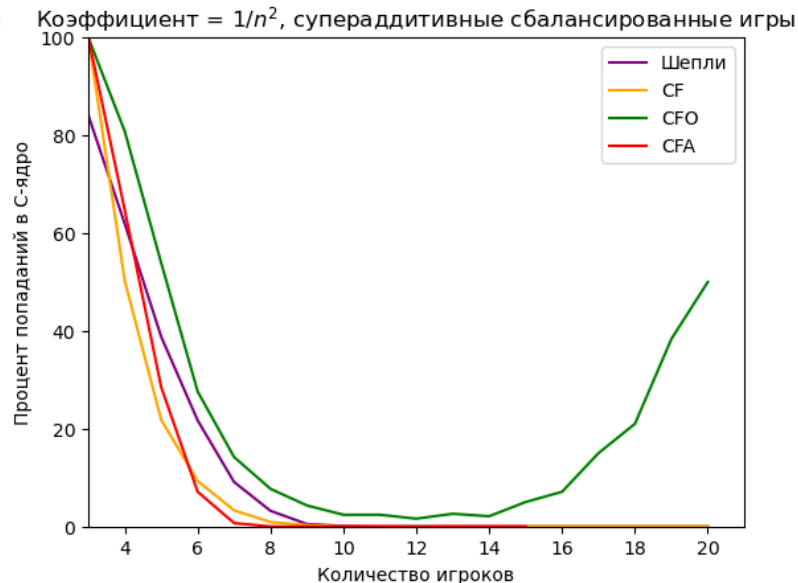
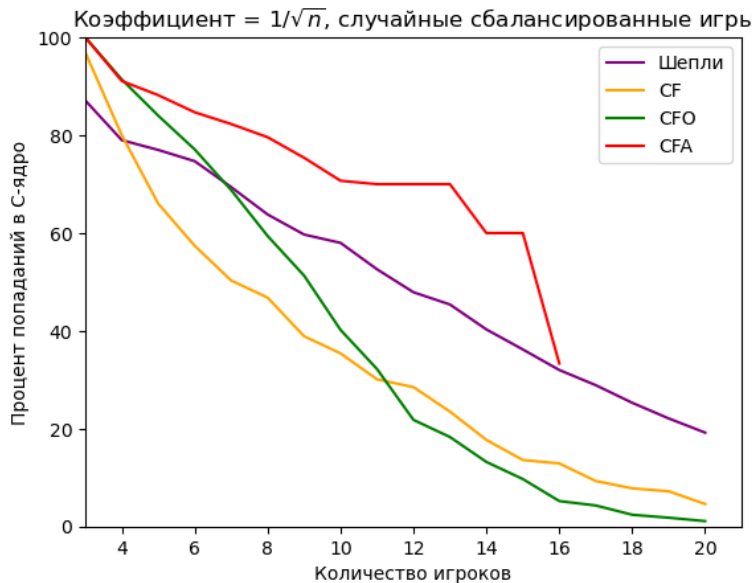
С коэффициентом $\frac{1}{n}$



- На случайных играх частота попаданий в С-ядро с ростом игроков падает до нуля у всех алгоритмов, но лучший результат показывает альтернативный алгоритм;
- На супераддитивных играх наилучший результат у оптимизированного алгоритма, на втором месте — вектор Шепли;

- Несмотря на меньшее отклонение от минимума, альтернативный алгоритм уступает базовому по частоте попаданий в С-ядро на супераддитивных играх.

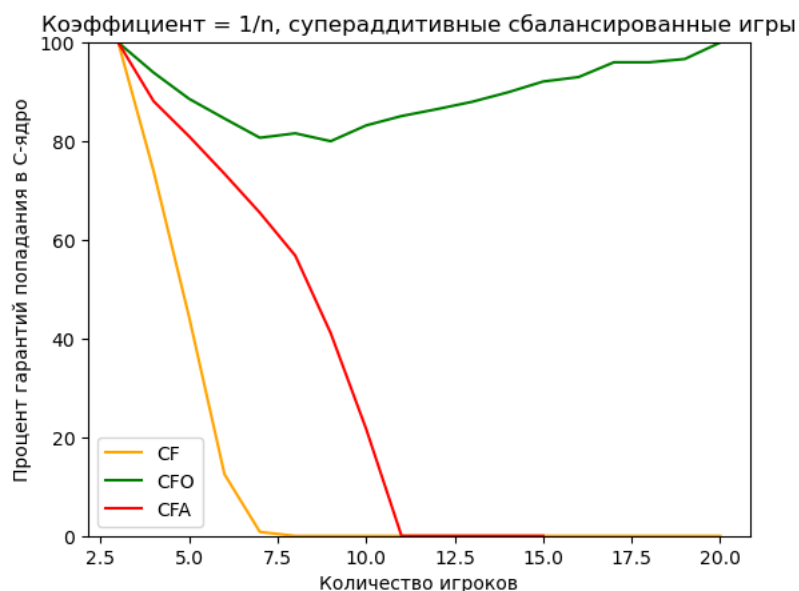
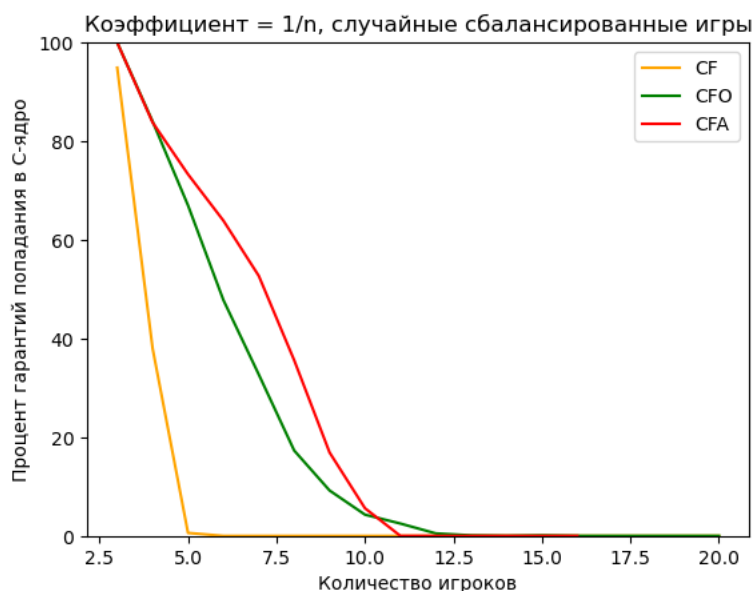
С другими коэффициентами



- На случайных играх с увеличенным коэффициентом результаты всех алгоритмов стали лучше. Однако у оптимизированного прирост результативности хуже, чем у остальных, из-за чего он уступает вектору Шепли и показывает частоту попадания на уровне базового алгоритма;
- На супераддитивных играх с уменьшенным коэффициентом у всех алгоритмов частота попадания резко стремится к нулю, хотя оптимизированному удаётся реабилитироваться ближе к 20 игрокам.

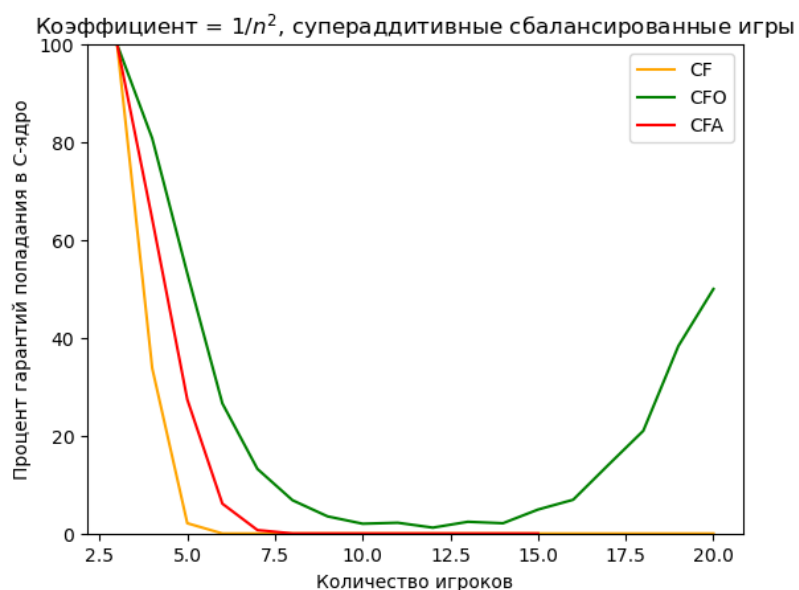
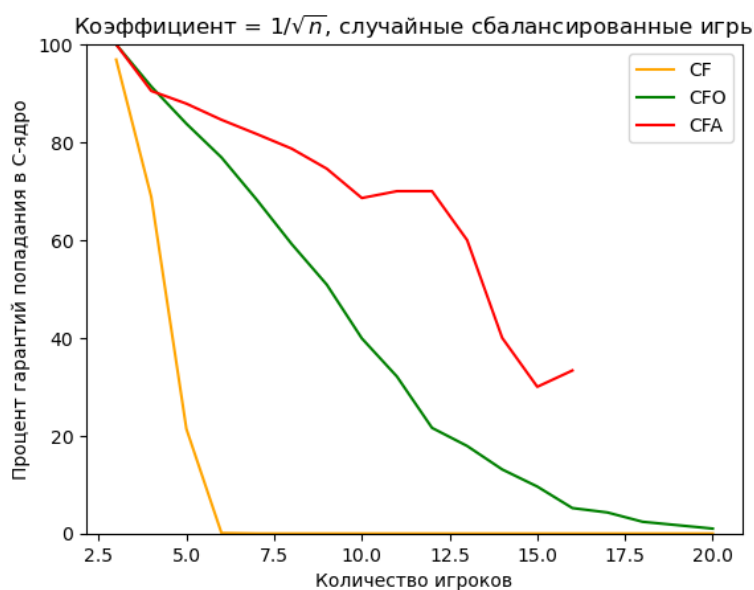
4.2.4 Гарантия попадания в С-ядро

С коэффициентом $\frac{1}{n}$



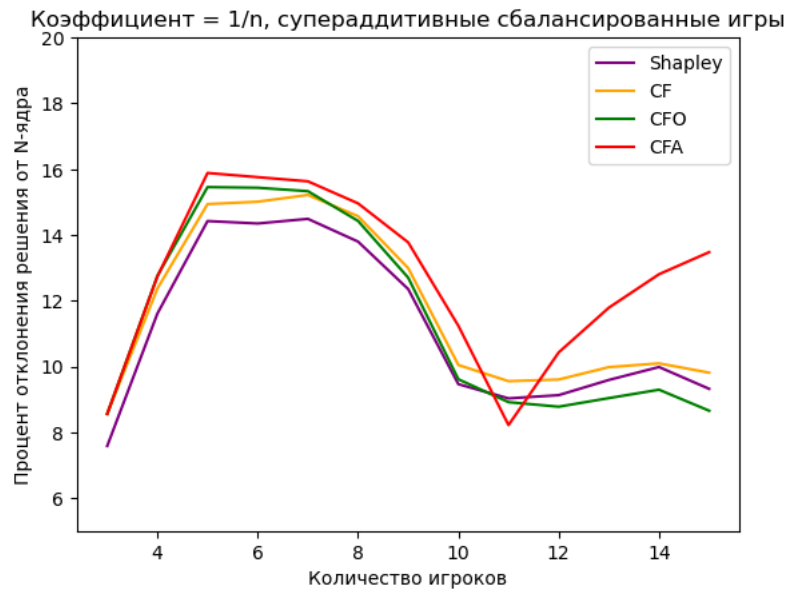
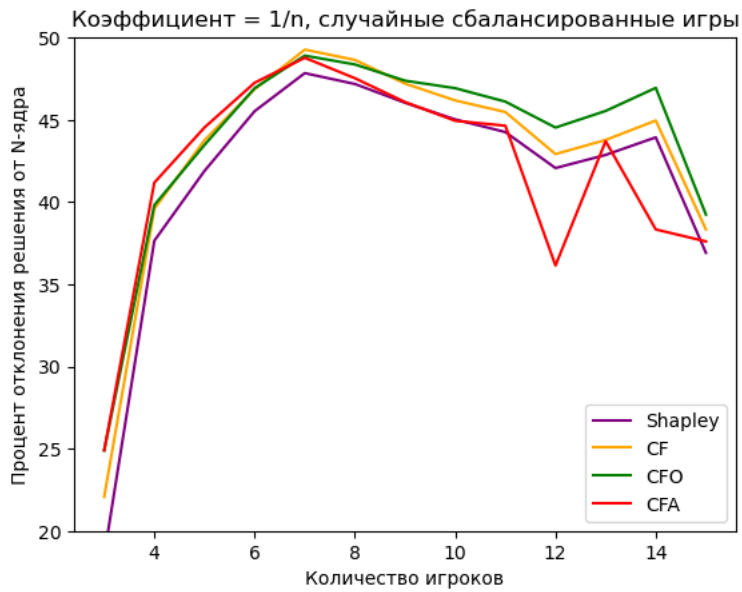
Результат схож с «процентом попадания в С-ядро», однако базовый алгоритм гораздо реже гарантирует попадание на супераддитивных играх. Следовательно, после уменьшения выигрышей в случае превышения $v(N)$ базовый алгоритм заметно чаще остальных оказывается в С-ядре.

С другими коэффициентами



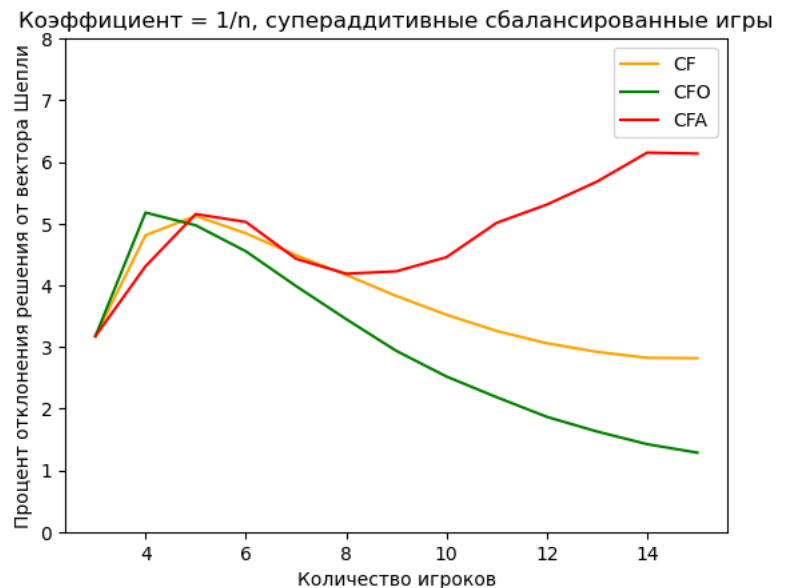
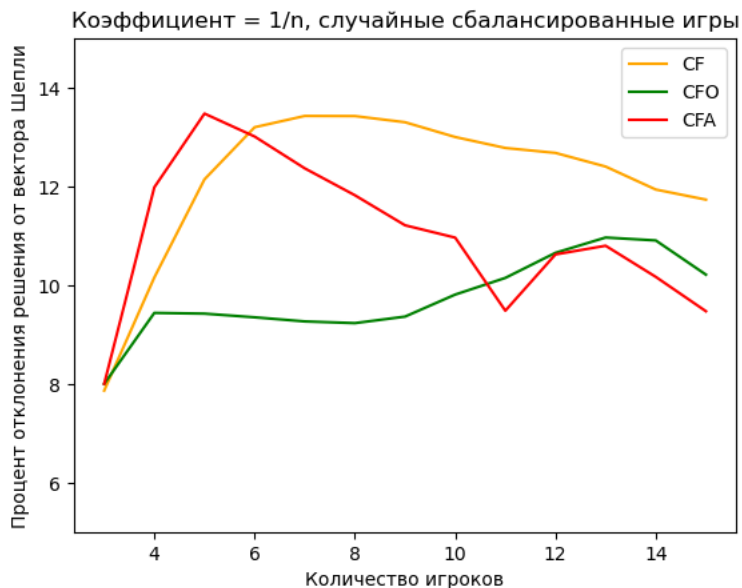
Здесь ситуация обратная, базовый алгоритм гораздо реже гарантирует попадание уже на случайных играх. В остальном графики повторяют «процентом попадания в С-ядро с другими коэффициентами».

4.2.5 Отклонение от N-ядра



- Все алгоритмы показывают практически идентичные результаты, немного выделяется только альтернативный.
- На случайных играх отклонение значительное, до 50%;
- На супераддитивных играх отклонение умеренное, не более 16%.

4.2.6 Отклонение от вектора Шепли



- Все алгоритмы обладают не слишком большим отклонением от вектора Шепли, особенно на супераддитивных играх;

- Ближе всего к вектору Шепли оптимизированный алгоритм.

Выводы

В ходе выполнения работы был предложен механизм построения распределения из С-ядра, изучены его свойства и посчитана асимптотическая сложность полного прохода по всем коалициям. А на основе изученных свойств разработаны и протестированы три алгоритма.

Результаты тестов:

- Оптимизированный алгоритм чаще всех остальных попадает в С-ядро на супераддитивных играх и показывает средний результат на случайных. При этом он обладает схожей с вектором Шепли скоростью работы и близким к нему решением;
- Альтернативный алгоритм чаще всех остальных попадает в С-ядро на случайных играх и реже всех на супераддитивных. Однако скорость работы у него даже хуже, чем у поиска N-ядра.
- Базовый алгоритм показывает сравнительно плохие результаты по попаданию в С-ядро, а гарантия попадания появляется ещё реже.

Заключение

Оптимизированный алгоритм показывает схожую с вектором Шепли скорость работы, чаще попадает в С-ядро и обладает близким к нему решением, но не имеет некоторых его аксиоматических свойств. Базовый алгоритм попадает в С-ядро реже, чем вектор Шепли, а альтернативный обладает скоростью работы даже ниже, чем у поиска N-ядра.

Рекомендуется применять оптимизированный алгоритм как резервный вариант, если вектор Шепли не попадает в С-ядро, а остальные не применять вовсе.

Список литературы

- [1] Gillies D. B. Some Theorems on n-Person Games. Ph.D. thesis. Princeton: Princeton University, 1954. 62 p.
- [2] Scarf H. The Core of an N person game // *Econometrica*. 1967. No 35. P. 50–69.
- [3] Peleg B., Sudhölter P. Introduction to the Theory of Cooperative Games. 2nd ed. Berlin, Heidelberg: Springer, 2007. 328 p.
- [4] Печерский С. Л., Яновская Е. Б. Кооперативные игры: решения и аксиомы. СПб.: Изд-во Европ. ун-та, 2004. 459 с.
- [5] Curiel I. Cooperative Game Theory and Applications: Cooperative Games Arising from Combinatorial Optimization Problems. Dordrecht: Kluwer Academic Publishers, 1997. 196 p.
- [6] Shapley L. S. A value for n-person games // *Contributions to the Theory of Games* / edited by H. W. Kuhn, A. W. Tucker. Princeton: Princeton University Press, 1953. Vol. 2. P. 307–318.
- [7] Maschler M., Peleg B., Shapley L. S. Geometric properties of the kernel, nucleolus and related solution concepts // *Mathematics of Operations Research*. 1979. Vol. 4. No 4. P. 303–338.
- [8] Schmeidler D. The nucleolus of a characteristic function game // *SIAM Journal on Applied Mathematics*. 1969. Vol. 17. No 6. P. 1163–1170.
- [9] Zakharov V. V., Akimova A. N. Nucleolus as a selector of subcore // *IFAC Proceedings Volumes*. 2000. Vol. 33. No 16. P. 675–680.
- [10] Парилина Е. М., Седаков А. А. Устойчивость коалиционных структур в одной модели банковской кооперации // *Математическая теория игр и её приложения*. 2012. Т. 4. Вып. 4. С. 45–62.

- [11] Frisk M., Göthe-Lundgren M., Jörnsten K., Rönnqvist M. Cost allocation in collaborative forest transportation // European Journal of Operational Research. 2010. Vol. 205. No 2. P. 448–458.